Minigraf 0507 and XY 4131

control with Arduino Nano Every

from an Atari XE/XL computer (SIO) and from a PC (USB)



By Zdeněk Kotrba

© 2026 · version 0.5.4 (firmware 0.5.1)

1. What the interface does

The Aritma Minigraf A 0507 is a Czechoslovak pen plotter from the 1980s. It has no intelligence of its own: it only accepts a phase number for each stepper motor (X1, X2, X4 and Y1, Y2, Y4) and a pen signal. The Arduino interface does this work for the computer – it accepts readable commands (simplified HPGL), converts lines into motor steps and handles acceleration and deceleration.

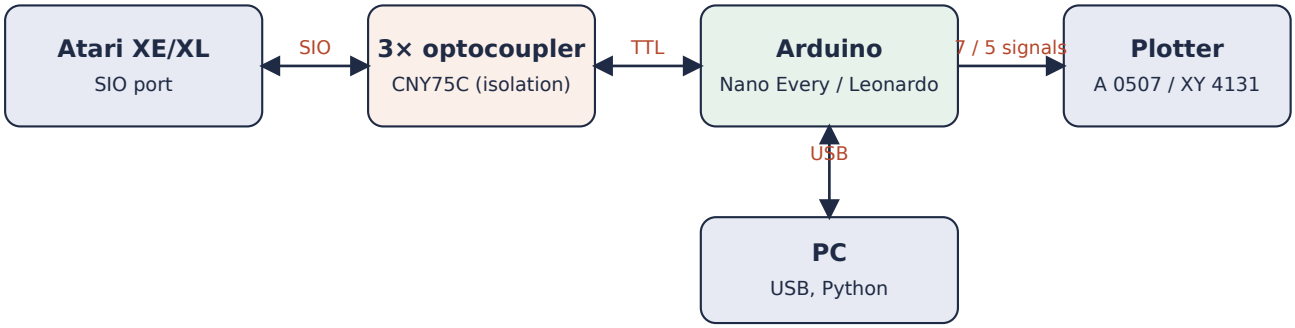


Fig. 1 – Overall setup

- **From the Atari** you draw with a plain LPRINT – the Arduino appears on the SIO bus as printer **P1:**.
- **From a PC** you draw with Python programs (`plot.py`, `text_plot.py`) or directly from the Arduino IDE Serial Monitor.
- The Atari is **galvanically isolated** by three optocouplers; the plotter connects to the Arduino directly.
- Transfer uses **handshaking**: the next command is sent only after the plotter has finished the previous one. Nothing gets lost.
- Since version 0.4.0 the same Arduino also drives the **Aritma XY 4131** plotter (chapter 11). Both plotters can be connected at the same time; the one selected with PL draws.

2. Parts list

Part	Qty	Note
Arduino Nano Every (ATmega4809)	1	recommended; alternative Arduino Leonardo / Micro (chapter 3)
Optocoupler CNY75C (or CNY75A/B)	3	OK1 Data Out, OK2 Command, OK3 Data In
Resistor 1 kΩ	3	optocoupler LED current
Resistor 2.2 kΩ	2	pull-up on OK1 and OK2 outputs
Resistor 470 kΩ	3	base–emitter, speeds up transistor turn-off
SIO connector, 13 pin (or a cable from an SIO device)	1	pins 3, 4, 5, 7, 10
Minigraf connector, 12 pin, 2.54 mm pitch	1	key at position 7, bottom row of contacts only
Optional: 7× 220 Ω, 100 µF + 100 nF	–	signal protection, Arduino supply decoupling

Power

- **Arduino** is powered from USB (PC or USB adapter). Do not power it from the plotter.
- **Minigraf** has its own **24 V** supply (about 30 W, supply rated at least 1.5 A). Supply minus is also the logic ground – pins 1 and 11 of the data connector.
- **The Atari side** (optocoupler LEDs) is powered from SIO pin 10 (+5 V).

Important: Atari ground (GND_A) and Arduino/plotter ground (GND_D) must **never be connected**. That is what the optocouplers are for.

3. Supported Arduino boards

The firmware `minigraf_sio.ino` detects the board from the processor. Both boards have a separate hardware UART (`Serial1` on D0/D1) for SIO, so USB stays free for the PC and debugging. All six motor phases are always on one processor port and change at the same instant.

Signal	Nano Every	Leonardo / Micro	Minigraf pin	Function
SIO Data Out (via OK1)	`D0 (RX1)`	`D0 (RX1)`	–	data from Atari
SIO Data In (via OK3)	`D1 (TX1)`	`D1 (TX1)`	–	replies to Atari
SIO Command (via OK2)	`D2`	`D2`	–	start of SIO command
X1	`A0`	`A0`	10	X phase, bit 0
X2	`A1`	`A1`	9	X phase, bit 1
X4	`A2`	`A2`	8	X phase, bit 2
Y1	`A3`	`A3`	4	Y phase, bit 0
Y2	`A6`	`A4`	3	Y phase, bit 1
Y4	`A7`	`A5`	2	Y phase, bit 2
P (pen)	`D10`	`D10`	5	H = pen down
GND	`GND`	`GND`	1, 11	common ground with plotter

	Nano Every (recommended)	Leonardo / Micro
Processor	ATmega4809, 48 kB flash, 6 kB RAM	ATmega32U4, 32 kB flash, 2.5 kB RAM
Board package in Arduino IDE	Arduino megaAVR Boards (clones: MegaCoreX, Pinout "Nano Every")	Arduino AVR Boards (built in)
Motor phase port	PORTD (A0–A3, A6, A7)	PORTF (A0–A5)
Note	A6/A7 are full digital pins on the Nano Every	Micro uses the same pin labels

Minigraf connector

The cable connects to the plotter with the same type of edge connector used by ZX Spectrum bus peripherals, with **2.54 mm** pitch. It has only **12 positions**, and **position 7 is a key**. The plotter PCB has contacts on the bottom side only, so **only the bottom row** of the connector is used.



Fig. – Connector position numbering (key at position 7 shown in black)

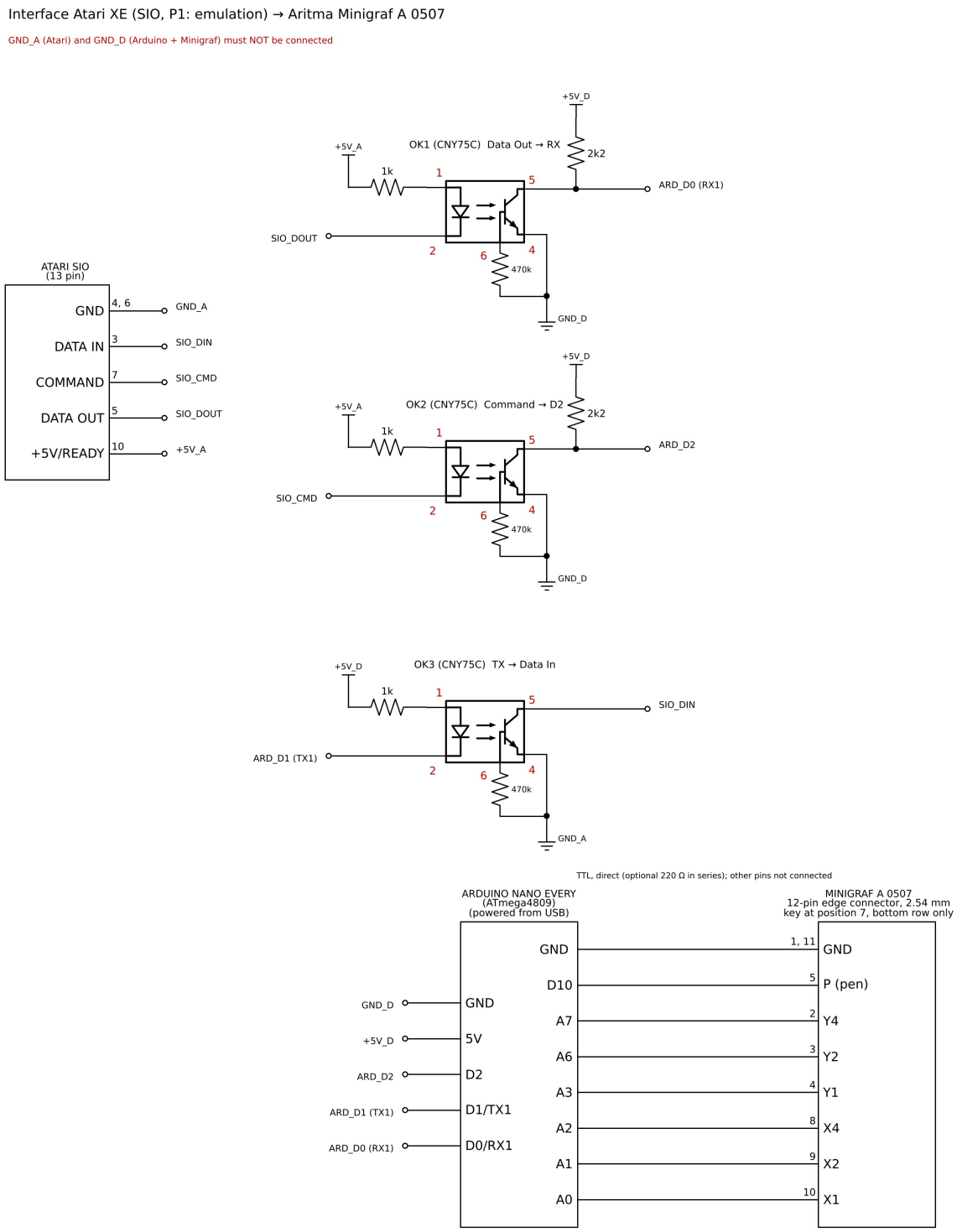
	12	11	10	9	8	7	6	5	4	3	2	1
signal:	**nc**	**GND**	**X1**	**X2**	**X4**	*key*	**nc**	**P**	**Y1**	**Y2**	**Y4**	**GND**
Nano Every:		GND	A0	A1	A2			D10	A3	A6	A7	GND

bottom row of plotter contacts (position 7 = key, no contact)

Fig. – Signals at each position and the matching Arduino Nano Every pins

Caution: the key **counts as pin 7** in the numbering even though it has no contact. Contacts to the left of the key are therefore 8 to 12, not 7 to 11. Numbers moulded on some connectors do not match this numbering – check with an ohmmeter that pins 1 and 11 are plotter ground before connecting.

4. Schematic

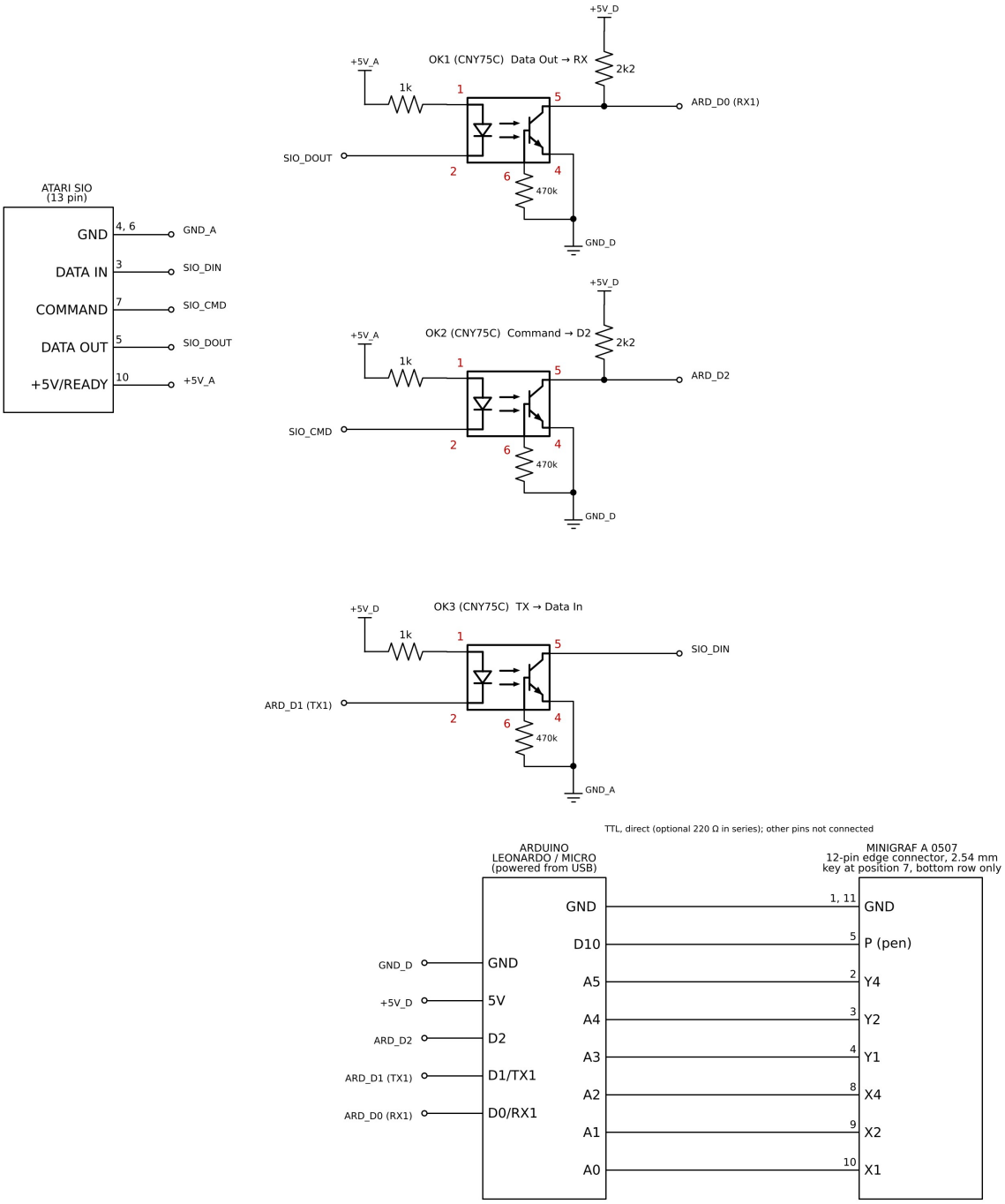


Note: +5V_A = SIO pin 10 (Atari), +5V_D = Arduino 5 V. All optocouplers are non-inverting.
If D0 is not below 0.8 V while the Atari transmits, reduce 1k at OK1/OK2 to 680 Ω.

Fig. 2 – Wiring with Arduino Nano Every

Interface Atari XE (SIO, P1: emulation) → Aritma Minigraf A 0507

GND_A (Atari) and GND_D (Arduino + Minigraf) must NOT be connected



Note: +5V_A = SIO pin 10 (Atari), +5V_D = Arduino 5 V. All optocouplers are non-inverting.
If D0 is not below 0.8 V while the Atari transmits, reduce 1k at OK1/OK2 to 680 Ω.

Fig. 3 – Alternative wiring with Arduino Leonardo / Micro (only Y2 = A4 and Y4 = A5 differ)

5. Software installation

Arduino firmware

- In the Arduino IDE open the **Boards Manager**, search for **megaAVR** and install **Arduino megaAVR Boards**.
- Select **Tools → Board → Arduino megaAVR Boards → Arduino Nano Every** and the correct port.
- Open `minigraf_sio.ino` and upload it.
- The message `jtagmkII_initialize(): Cannot locate "flash" and "boot" memories` is normal and harmless on the Nano Every.
- Nano Every clone with a CH340 USB chip: use **MegaCoreX** instead, board ATmega4809 and **Pinout: Nano Every**.

Python on the PC (Windows)

- Install Python 3 and run in the **command prompt** (not in the Python window with >>>):

```
py -m pip install pyserial
py -m pip install matplotlib      (only for --preview)
```

- Put `plot.py`, `text_plot.py` and `hershey_script.py` in the same folder, e.g. `C:\plotter`.
- The default port is **COM4**. Give another port before the file name or with `--port`.
- The **Serial Monitor must be closed** while plotting from Python – only one program can open the port.

6. Coordinates and plotting area

Parameter	Value
Unit	1 step = 0.125 mm (8 steps = 1 mm)
Absolute X position	0 = left carriage stop (after HM), limit 1580 steps (verified, changed by LM)
Absolute Y position	0 = paper loaded with its top edge at the cut-outs under the feed wheels, limit 2284 steps (verified)
Start position	X = 60, Y = 144 – the head moves here after HM
Drawing area from start	X 0 – 1520 steps (190 mm), Y 0 – 2140 steps (267.5 mm)
A4 margins (measured)	left and right 10 mm, top and bottom 15 mm
Drawing point 0,0	set by IN; – the PC programs send HM;IN; automatically
Centre of area (after HM;IN;)	X = 760, Y = 1070

Homing – HM (from version 0.3)

The plotter has no limit switch, but the carriage has a mechanical stop. `HM;` lifts the pen and drives the head left by the full width plus 200 extra steps. At the stop the stepper simply slips (harmless to the plotter), so the head always ends up exactly at the left, wherever it was. The firmware sets X = 0 there and moves to the start position. The Y axis (paper) has no stop: the paper position at the moment of HM is Y = 0, so A4 paper is **always loaded with its top edge at the cut-outs under the feed pinch wheels**, and HM is sent right after. The start position X = 60, Y = 144 gives A4 margins of 10 mm at the sides and 15 mm at the top. The Y limit 2284 leaves 15 mm at the bottom too – so the paper does not slip out of the feed wheels (with a longer area it slipped out by about 3 mm).

From then on the firmware knows the **absolute position** from the stop and checks the area limits against it – `IN;` only shifts the drawing origin, the limits stay in place. Limits are set with `LM x,y;` and stored in EEPROM, so they survive power-off.

The PC programs send `HM;` automatically before every drawing (disable with `--nohome`). From the Atari, start your program with `LPRINT "HM;IN;"`.

7. Command reference

Commands are the same from the Atari, the Serial Monitor and in PC files. Each command starts with two letters, numbers are separated by commas and commands by semicolons. Several commands may be on one line.

Command	Meaning	Example
`IN;`	initialise: current position = 0,0, pen up, absolute coordinates	`IN;`
`PU x,y;`	pen up and move to x,y (several points allowed)	`PU760,1070;`
`PD x,y,...;`	pen down and draw through the given points	`PD800,100,800,900;`
`PA;`	following coordinates are absolute (default)	`PA;PU0,0;`
`PR dx,dy;`	following coordinates are relative to current position	`PR80,0;`
`CI r[,n];`	circle of radius r around current position, n segments (default 72)	`PU750,1050;CI400;`
`VS v;`	speed in mm/s; VS; = default (60 mm/s)	`VS40;`
`PT d,u;`	pen delay in ms after pen down / pen up; PT; = default (30 / 70)	`PT20,70;`
`HM[x[,y]];`	home to left stop (X = 0), paper = Y 0, then move to start (default 60,144)	`HM;`
`LM x[,y];`	area limits in steps from the stop, stored in EEPROM; LM; = print (default 1580,2284)	`LM1580,2284;`
`AC s,r;`	acceleration: start delay s µs, shortened by r µs per step; AC; = default (6000 / 250)	`AC5000,300;`
`DG0; / DG1;`	diagonal lines at full speed / slowed ×1.4; DG; = default (0)	`DG1;`
`PL0507; / PL4131;`	plotter selection, saved to EEPROM; PL; = print (since 0.4.0, chapter 11)	`PL4131;`
`SI n;`	text height in mm (capitals), default 6 (since 0.5.0)	`SI8;`
`FN0; / FN1;`	block / script (handwriting-like) font	`FN1;`
`DI0; / DI1;`	text along the X axis / rotated 90° along the Y axis	`DI0;`
`LBtext`	writes text from the pen position; text runs to the end of the line or CHR$(3) (chapter 8)	`LBHello`
`?`	Serial Monitor only: firmware version, plotter and status (position from origin and absolute, limits, pen, speed, AC, DG, PT)	`?`

Useful moves

What	Command
Home to left stop and start position	`HM;`
Head back to drawing origin	`PU0,0;`
Head to centre of area (after HM;IN;)	`PU760,1070;`
Move 10 mm right (without drawing)	`PU;PR80,0;PA;`
Square 50 × 50 mm	`PU200,200;PD600,200,600,600,200,600,200,200;PU;`
New origin at current position	`IN;`

Tuning acceleration (from version 0.2)

Every line starts slowly, accelerates to the set speed and slows down before its end. From version 0.3.2 the default is the verified setting `AC6000,250` (full speed after 2 mm) and diagonal lines run at full speed (`DG0`). For a gentler setting on an older or worn plotter use `AC9000,150;DG1;`. You can detect lost steps when the head does not return exactly to the origin after `PU0,0;`.

Setting	Start	Full speed after	Note
`AC9000,150;`	14 mm/s	46 steps (5.8 mm)	gentlest (default up to 0.3.1)
`AC6000,250;` (default)	21 mm/s	16 steps (2 mm)	verified: print looks good
`AC5000,300;`	25 mm/s	10 steps (1.2 mm)	fast; check return to origin

Firmware defaults (Minigraf)

Constant in minigraf_sio.ino	Value	Meaning
`STEP_US`	`2083`	speed 60 mm/s (= 125000 / speed)
`START_US / RAMP_US`	`6000 / 250`	start and stop of every line (changed by AC, verified)
`DIAG_SLOW`	`false`	diagonal lines at full speed (changed by DG, verified)
`FW_VERSION`	`"0.5.1"`	firmware version, printed at start-up and by ?
`X_MAX_DEF / Y_MAX_DEF`	`1580 / 2284`	default area limits from the stop (overridden by LM stored in EEPROM)
`HOME_X / HOME_Y`	`60 / 144`	start position after HM (A4: 10 mm left margin, 15 mm top margin)
`HOME_DIR`	`−1`	direction to the left stop (+1 = opposite)
`HOME_EXTRA / HOME_US`	`200 / 2500`	extra steps beyond the width / homing speed (~50 mm/s)
`MIN_STEP_US`	`1500`	maximum allowed speed about 83 mm/s
`PEN_DOWN_MS / PEN_UP_MS`	`30 / 70`	pen delay (verified with a 0.5 mm marker)
`PEN_IDLE_MS`	`1000`	pen down and no command for this long → firmware lifts it (no ink blot on a dropout)
`PEN_DOWN_LEVEL`	`HIGH`	P signal level for pen down
`X_DIR / Y_DIR`	`+1 / +1`	change to −1 to reverse an axis
`PRN_TIMEOUT_S`	`255`	how long the Atari waits for a line to finish

8. Plotting from the Atari

On the SIO bus the Arduino behaves as printer **P1:**. You draw with a plain LPRINT in Atari BASIC (or by writing to device `P:` from other programs). After each line the Atari waits until the Arduino confirms the drawing is finished – no timing needed.

Example Atari BASIC program

```
10 REM MINIGRAF 0507 - DEMO
20 LPRINT "HM;IN;"
30 REM FRAME
40 LPRINT "PU100,100;PD1400,100,1400,2000,100,2000,100,100;"
50 REM CIRCLE IN THE MIDDLE
60 LPRINT "PU750,1050;CI400;"
70 REM VERTICAL LINES
80 FOR I=0 TO 20
90 LPRINT "PU";100+I*60;",300;PD";100+I*60;",900;"
100 NEXT I
```

```
110 LPRINT "PU0,0;"
```

Quick commands from the Atari

Atari BASIC	Result
`LPRINT "HM;IN;"`	home to the stop, start position = origin
`LPRINT "PU0,0;"`	head back to drawing origin
`LPRINT "VS40;"`	slower drawing (40 mm/s)
`LPRINT "VS;PT;"`	back to default speed and pen delay
`LPRINT "AC6000,250;DG0;"`	faster start and diagonal lines at full speed
`LPRINT "PU750,1050;CI200;"`	circle 50 mm in diameter in the middle

Text directly from the Atari (since firmware 0.5.0)

The firmware contains both fonts of `text_plot.py` – block and script – so the Atari sends only the text and the Arduino draws it. Text starts on the **baseline** at the last pen position and the pen ends up after the text. Everything after LB up to the end of the line is text (no semicolon at the end); to continue with further commands, end the text with `CHR$(3)`.

```
10 REM TEXT FROM THE ATARI
20 LPRINT "HM;IN;SI8;FN1;"
30 LPRINT "PU100,1900;LBAhoj, tohle pi's^e Atari."
40 LPRINT "PU100,1780;LBZdene^k Kotrba"
50 LPRINT "FN0;SI5;PU100,1660;LBBlokove' pi'smo 5 mm"
60 LPRINT "LBmore text";CHR$(3);"PU0,0;"
```

In the text	Result
`a' e' i' o' u' y' (also capitals)`	á é í ó ú ý – acute
`c^ d^ e^ n^ r^ s^ t^ z^ (also capitals)`	č ď ě ň ř š ť ž – caron
`u* U*`	ů Ů – ring
`\' \^ \*`	literal character, e.g. I\'m
`Kamenický (bytes $80–$AF)`	Czech letters directly; only Š ($9B) is the Atari end of line – write S^
`UTF-8 (Serial Monitor)`	Czech letters directly, e.g. LBPříliš žluťoučký kůň

`SI` is the capital height in mm; script lower-case letters are small, so use about 1.5× the size. Space lines at 1.8× the text size (Minigraf: SI8 = 64 steps, next line 115 steps lower; XY 4131: SI8 = 80 steps). `DI1;` rotates the text by 90° (along the paper, like the lettering from `text_plot.py`).

Atari tips

- One LPRINT may contain several commands; longer lines are split into several SIO records automatically.
- Atari BASIC prints numbers without spaces, so `LPRINT "PU";X;",";Y;";"` works directly.
- If a single line took longer than about 4 minutes, the Atari would report error 138 – split such a line into several LPRINT.
- While commissioning, the Serial Monitor shows `SIO: status` and `SIO: zapis OK` for each LPRINT.

9. Plotting from a PC

9.1 Serial Monitor (manual commands)

In the Arduino IDE open the Serial Monitor, set **115200 baud** and **Newline**. Each line you type is drawn and the Arduino replies `hotovo` ("done"). Board LED blinking = program running, steady = drawing.

9.2 Plotting an HPGL file – plot.py

```
py plot.py drawing.hpgl              (HPGL from Inkscape, port COM4)
py plot.py napis.hpgl --scale 1      (file already in plotter steps)
py plot.py COM7 drawing.hpgl         (another port)
py plot.py drawing.hpgl --dry        (only print what would be sent)
```

Option	Meaning
`--scale 0.2`	coordinate scale; 0.2 = HPGL from Inkscape (40 units/mm), 1 = plotter steps
`--speed 40`	speed in mm/s for this drawing
`--pen 0,70`	pen delay in ms (after pen down, after pen up)
`--accel 6000,250`	acceleration for this drawing (same as AC command)
`--diag 0`	diagonal lines at full speed (1 = slowed)
`--nohome`	do not home (HM) first; draw from where the head is
`--dry`	send nothing, only print the converted commands
`--plotter 4131`	draw for this plotter (0507 / 4131); switches the Arduino if another one is selected
`--select 0507`	only switch the plotter in the Arduino and exit (no file)
`--version`	print program version; the firmware version is printed automatically after connecting

Plotting from Inkscape: File → Save As → *HP Graphics Language (*.hpgl)*, resolution 1016 dpi, leave "Center zero point" unticked.

9.3 Lettering and text – text_plot.py

The program has its own stroke font (upper and lower case, digits, punctuation, Czech diacritics) and since version 0.4.3 also a **handwriting-like script** (option `--font psane`). Three modes:

```
Large lettering across the whole width (A4 landscape):
  py text_plot.py "HELLO" --frame --star --bold 3 --send

Normal text in lines, wrapped automatically ( | = new line):
  py text_plot.py "Hello,|this is Minigraf." --size 5 --send
  py text_plot.py --file letter.txt --size 4 --portrait --send

Test page (6 / 4 / 3 / 2 mm):
  py text_plot.py --test --portrait --send
```

Option	Meaning
`--send`	send to the plotter right away (otherwise only creates napis.hpgl)
`--size 5`	capital letter height in mm – normal text mode
`--portrait`	A4 portrait like a printer (lines along the short side)
`--file letter.txt`	text from a file (save as UTF-8)
`--frame / --star`	double frame / eight-pointed star above large lettering
`--bold 1–4`	bold: letters are traced several times with a small offset

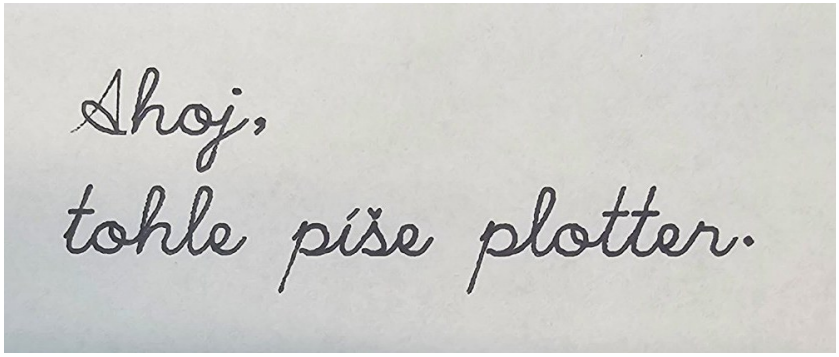
Option	Meaning
`--font psane`	handwriting-like script (Hershey script font with added Czech diacritics); needs hershey_script.py in the same folder. Default --font blok
`--margin 8`	margin inside the drawing area in mm (default 8 for lettering, 5 for text; the area already has A4 margins)
`--flip`	rotate the drawing by 180° if it comes out upside down
`--preview`	save a .png preview (needs matplotlib)
`--speed / --pen / --accel / --diag / --nohome / --port`	same as plot.py
`--version`	program version

Script font (--font psane)

```
py text_plot.py "Zdeněk Kotrba" --font psane --frame --star --bold 2 --send
py text_plot.py --file letter.txt --font psane --size 6 --portrait --send
```



Script font sample (program preview). --size sets the capital height; script lower-case letters are smaller, so use about 1.5× the size you would use for the block font.



Script font plotted on the Minigraf A 0507: text_plot.py --font psane

Recommended letter size by pen

Pen line width	Smallest well-readable letters
0.5 mm	about 5 mm
0.3 mm	about 3 mm
0.1–0.2 mm	about 2 mm

10. Example results



Fig. 4 – Large lettering with frame and star, speed 80 mm/s, 0.5 mm marker.
`py text_plot.py "ZDENĚK 0507" --frame --star --bold 3 --send --speed 80`

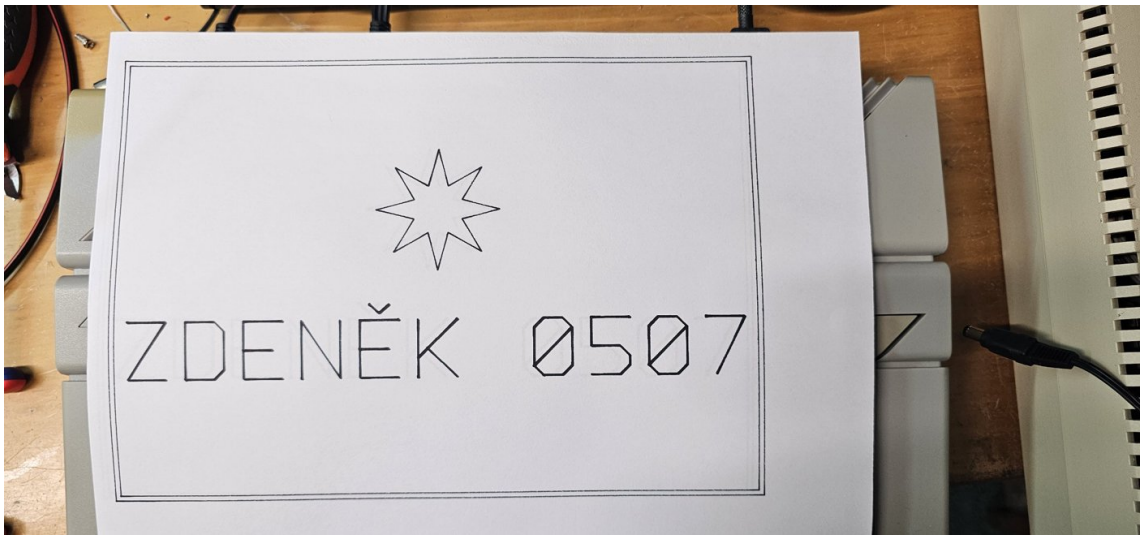
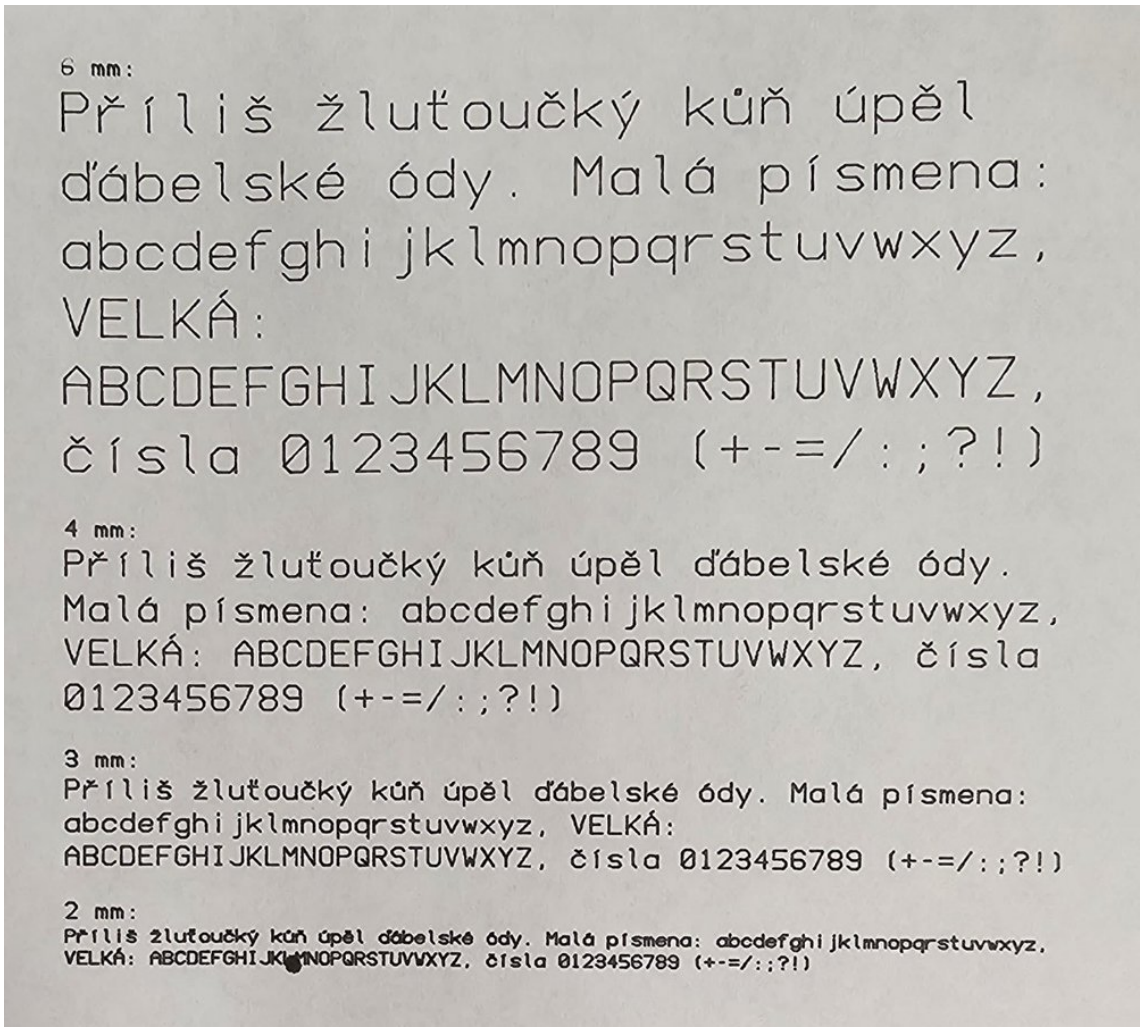


Fig. 5 – The same drawing from the first tests, view of the plotter.

Fig. 6 – Test page

`py text_plot.py --test --portrait --send` – firmware and programs 0.3.4, default settings (speed 60 mm/s, acceleration AC6000,250, diagonal lines DG0, pen delay PT30,70), 0.5 mm marker, 419 s.



- 6, 4 and 3 mm: clean and easy to read, loops close, diacritics in place, lines are straight.
- 2 mm: readable, but a 0.5 mm marker is at its limit – letters look bold. A 0.3 mm marker will be clean.
- **Why the ink blot appeared** (last line near "JKLM"): at line 2058 of 2198 one command was lost on USB (`PD368,836,379,836;`), so the Arduino did not reply "hotovo" (done). Version 0.3.4 waited 20 s before querying the Arduino and resending the command – and all that time the pen stayed down on the paper, so the ink soaked into one spot. The drawing then finished correctly ("Znovu posláno řádků: 1" = 1 line resent).
- **Fix in version 0.3.5:** the firmware lifts the pen by itself when no command arrives for 1 s, and `plot.py` queries after only 3 s. A dropout now delays the drawing by just a few seconds and leaves no blot.

Fractals (fraktaly.py, since version 0.5.2)

`fraktaly.py` draws ten fractals described by L-systems. The figure is scaled to the whole drawing area, rotated to be as large as possible, and most of them are drawn in **one stroke** – the pen goes down only at the start. The plant and the bush have several strokes. `--order` changes the level of detail; the program warns when segments would be shorter than 1 mm. All options are described in chapter 13.

```
py fraktaly.py --list                 overview
py fraktaly.py koch --send            Koch snowflake
py fraktaly.py drak --send            dragon curve
py fraktaly.py hilbert --order 5      coarser Hilbert curve (file and preview only)
py fraktaly.py strom --plotter 4131 --send
```

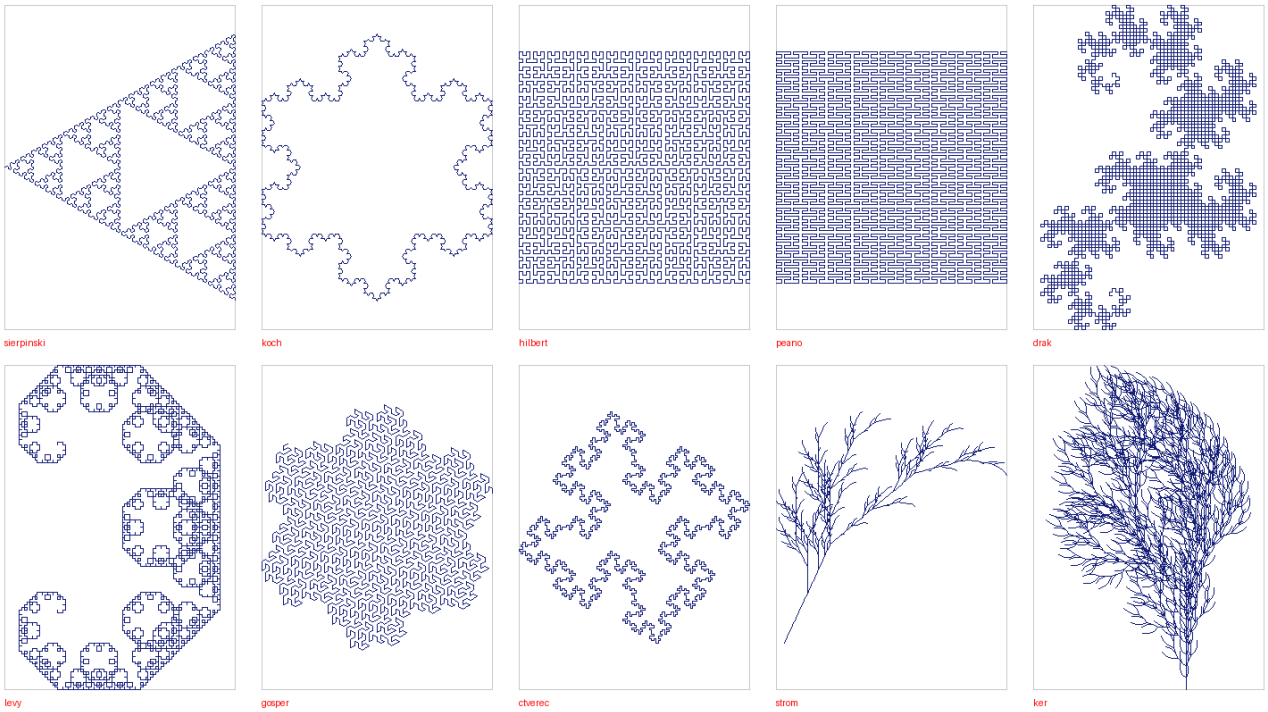


Fig. 7 – Fractal previews at default order on the Minigraf area: sierpinski, koch, hilbert, peano, drak (dragon) / levy, gosper, ctverec (quadratic Koch), strom (plant), ker (bush)

11. The Aritma XY 4131 plotter

Unlike the Minigraf, the XY 4131 (and the similar XY 4130) has its own stepper electronics. It is controlled by five 5 V logic signals: **STEP** (one pulse = 0.1 mm), **X/Y** (which axis), **+/−** (direction), **PEN** and ground. The pen carriage moves across (X axis), the paper is moved by the roller (Y axis). The plotter steps only one axis at a time, so for diagonal lines the firmware alternates the axes step by step.

11.1 Wiring

Signal	Nano Every / Leonardo	INPUT pin	Meaning (verified)
PEN	D6	1	1 = pen down (pulled by the coil), 0 = up (spring)
STEP	D3	2	idle 1, pulse to 0 (20 µs) = one 0.1 mm step
X/Y	D5	3	0 = pen carriage (X axis), 1 = paper (Y axis)
+/−	D4	4	carriage: 1 = right, 0 = left; paper: 1 = forward, 0 = back
READY	D7 (not used)	5	open collector; always 0 on this unit, not needed by the firmware
GND	GND	6	plotter ground
–	–	7	not connected

Connector: the plotter has a round 7-pin socket with pins labelled **INPUT** at the back. The cable mates with a round TESLA **WK 180 35** plug with sockets and a threaded coupling nut (the type is stamped on the plug body). Pin numbers are moulded in the blue insert; the plug seen from the front is mirrored against the plotter socket, so follow the numbers on the insert when soldering. Pin 7 (centre) is not connected.



Fig. 9 – WK 180 35 cable plug from the front, its marking and the INPUT socket on the plotter

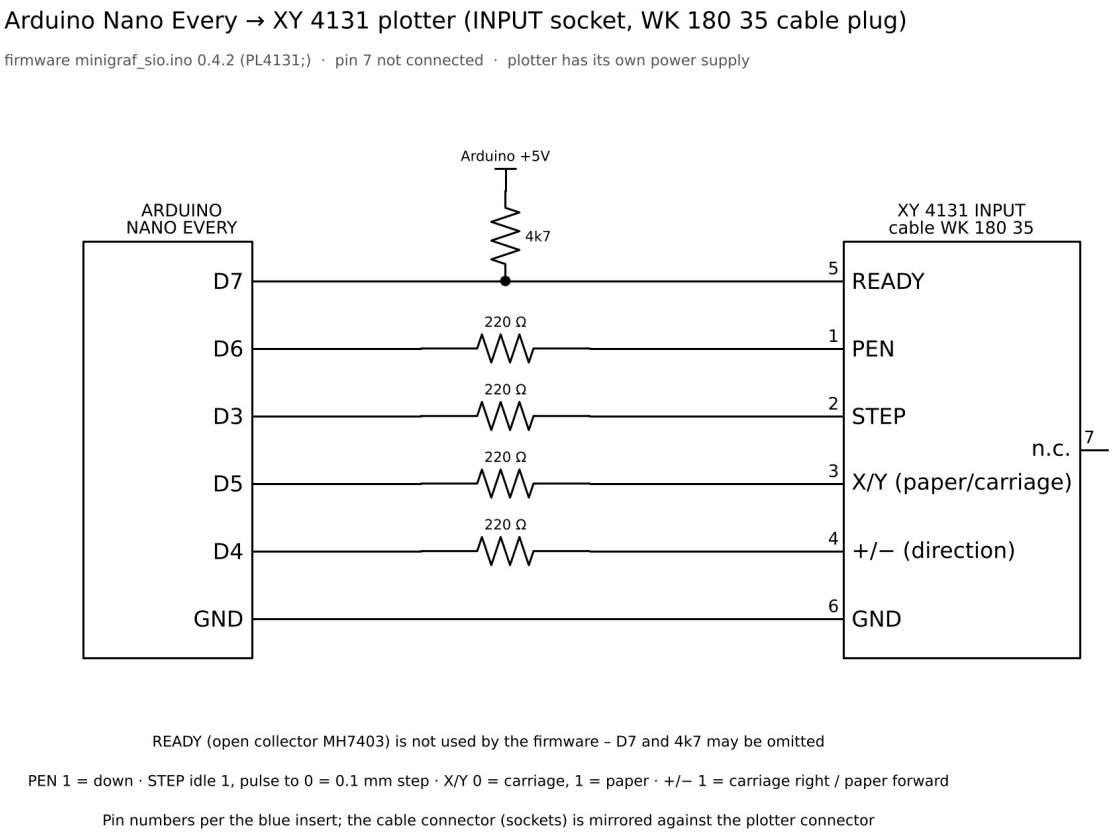


Fig. 10 – XY 4131 wiring (220 Ω resistors recommended)

Check before the first drawing: the plotter has a **pen switch** on its panel – when it is off, the pen ignores the PEN signal. The carriage end contacts must spring back freely: **a stuck left contact also blocks moving right** (the carriage then does not move even after power-cycling the plotter until the contact is freed).

11.2 Selecting the plotter and first start

```
PL4131;     switch to XY 4131 (saved, survives power-off)
PL0507;     back to Minigraf A 0507
PL;         print the selected plotter (also shown by ? and the start-up message)
```

The plotter can be switched in three ways – the choice is always saved to the Arduino EEPROM:

```
Serial Monitor:     PL0507;   or   PL4131;
PC command line:    py plot.py --select 0507      (other port: py plot.py COM7 --select 4131)
Atari BASIC:        LPRINT "PL0507;"   or   LPRINT "PL4131;"
```

After switching, the firmware loads that plotter's default speed, acceleration, pen delays and limits and treats the position as unknown – so always send `HM;` before drawing (the PC programs do it themselves). `HM;` lifts the pen, drives the carriage left over the whole travel (the end contact stops it), after a 200 ms pause sets X = 0, takes the current paper position as Y = 0 and moves to the start X = 100 (10 mm).

Parameter	XY 4131
Unit	1 step = 0.1 mm (10 steps = 1 mm), measured on both axes
Carriage travel	about 1800 steps between the end contacts; X limit = 1780
Paper	Y limit = 2600 (260 mm) from the position at HM; positive Y = paper moves back
Start after HM	X = 100, Y = 0
Area for text_plot.py	1600 × 2400 steps (160 × 240 mm) from the start
Speed	default 50 mm/s (VS; = 50), maximum 66 mm/s

Parameter	XY 4131
Acceleration	AC5000,250
Pen	PT30,70 – the coil pulls it down at once, the spring lifts it and it settles

11.3 Drawing from a PC

`plot.py` and `text_plot.py` ask the Arduino which plotter is selected after connecting and use the right scale and area automatically. With --plotter 0507 or --plotter 4131 (since version 0.4.4) they switch the Arduino to that plotter if another one is selected. Load the paper so that the pen starts about 2 cm from the edge with at least 25 cm of paper beyond it.

```
py text_plot.py "Zdeněk XY4131" --frame --star --bold 3 --send
py text_plot.py --test --send --speed 50
py plot.py picture.hpgl                (HPGL from Inkscape: scale 0.25 automatically)
py text_plot.py "HELLO" --plotter 4131 --send   (switches to XY 4131 and draws)
py text_plot.py "HELLO" --plotter 4131 (file only, no connection)
```

11.4 Drawing from the Atari

Same as with the Minigraf, only the unit is 0.1 mm. Select the plotter once from the Serial Monitor (`PL4131;`) or from the Atari: `LPRINT "PL4131;HM;IN;"`.

```
10 LPRINT "PL4131;HM;IN;"
20 LPRINT "PU100,100;PD1500,100,1500,2300,100,2300,100,100;"
30 LPRINT "PU800,1200;CI500;"
40 LPRINT "PU0,0;"
```

Choosing the plotter at the start of a program (then use the unit of the selected plotter):

```
10 ? "PLOTTER: 1 = MINIGRAF 0507, 2 = XY 4131";:INPUT P
20 IF P=1 THEN LPRINT "PL0507;HM;IN;"
30 IF P=2 THEN LPRINT "PL4131;HM;IN;"
```

11.5 XY 4131 firmware defaults

Constant in minigraf_sio.ino	Value	Meaning
`XY_X_DIR / XY_Y_DIR`	`+1 / −1`	axis directions (with XY_Y_DIR = +1 the drawing came out mirrored)
`XY_X_MAX_DEF / XY_Y_MAX_DEF`	`1780 / 2600`	area limits (changed by LM, stored separately per plotter)
`XY_STEP_US`	`2000`	speed 50 mm/s (= 100000 / speed)
`XY_START_US / XY_RAMP_US`	`5000 / 250`	acceleration and deceleration
`XY_HOME_X / XY_HOME_Y`	`100 / 0`	start after HM
`XY_HOME_EXTRA / XY_HOME_US`	`120 / 1800`	HM drives left X limit + 120 steps / speed
`XY_PEN_DOWN_MS / XY_PEN_UP_MS`	`30 / 70`	pen delays
`XY_PULSE_US`	`20`	STEP pulse width

11.6 Test program xy4131_test.ino

For bringing up a new unit the repository contains a separate program `xy4131_test.ino` (version 0.10). Lower-case commands drive single signals (`p1, a0, d1, s 1000, t 1500, h, ?`), upper-case commands draw like the main firmware. This is how the wiring, axis directions, the 0.1 mm step (1000 steps = 100 mm) and the carriage travel were verified.

12. Troubleshooting

Symptom	Cause and fix
Head stops and hums	motor is losing steps – lower the speed (--speed or VS), check the 24 V supply and mechanics
plot.py prints "bez odpovědi, posílám znovu" (no reply, resending)	USB communication dropout (up to 0.3.2 mainly an overflow of the 64-byte Arduino buffer, fixed in 0.3.4); plot.py recovers by itself (after 3 s it queries the Arduino and resends the line) and firmware 0.3.5 lifts the pen meanwhile, so no ink blot appears. If frequent: 100 µF + 100 nF on Arduino 5 V, shorter ground to the plotter, short USB cable, ferrite on USB. Since version 0.5.1 plot.py uses a line counter in the firmware to tell whether the command was lost (resends it) or only the reply (does not resend – it would draw an extra line). All communication is logged to `plot_log.txt` next to plot.py.
plot.py prints "odpověď se ztratila, řádek ale Arduino nakreslilo" (reply lost, line was drawn)	the Arduino executed the command, only "hotovo" got lost on USB; plotting continues. If frequent, send plot_log.txt for analysis
plot.py prints "Arduino: zahozen neuplny prikaz: ..." (incomplete command discarded)	only the beginning of the line reached the Arduino (characters lost); plot.py resends the line
plot.py reports "ARDUINO SE RESTARTOVALO" (restarted)	the Arduino restarted and the position is lost – run the drawing again (HM homes to the stop); usually caused by interference or a supply dip
could not open port / Access denied	Serial Monitor open or wrong port number
No module named serial	py -m pip install pyserial
Nothing happens from the Serial Monitor	set "Newline" and 115200 baud (bottom right)
Blobs at the start of lines	shorten delay after pen down: --pen 0,70 or PT0,70;
Start of line missing	lengthen delay after pen down: --pen 30,70
Tails or smudges between letters	lengthen delay after pen up: --pen 0,100
Drawing mirrored	in minigraf_sio.ino change X_DIR or Y_DIR to −1
Drawing upside down	option --flip
Pen does the opposite (PD lifts)	change PEN_DOWN_LEVEL to LOW
Nothing after LPRINT, monitor silent	check OK1 and OK2; at rest the status must show Command=H
"chyba souctu" (checksum error) in monitor	slow edges at OK1 – reduce 1 kΩ to 680 Ω
Atari reports error 138	Arduino not responding or one line takes longer than the timeout – split the line
Text lines are slanted	paper loaded skewed – align it with the roller
Marker smears even mid-line	squashed tip – replace the marker
HM moves right instead of left	switch the plotter off at once; set HOME_DIR = +1 in minigraf_sio.ino
Head does not reach the right edge	limit too small: LM; prints it, LM1580,2284; sets it
Paper slips out of the feed wheels	Y limit too large: reduce it, e.g. LM1580,2250;
Drawing is shifted on the paper in Y	paper was not loaded at the cut-outs, or moved after HM – reload it and send HM

Symptom	Cause and fix
Small text is drawn slowly	faster start: --accel 6000,250 or AC6000,250;
plot.py reports firmware "neznámá" (unknown)	Arduino has firmware older than 0.2 – upload the new one
Firmware does not know HM	Arduino has firmware older than 0.3 – upload the new one, or plot with --nohome
XY 4131: pen does not react	switch on the pen switch on the plotter; check D6 → pin 1 (about 4.4 V after PD)
XY 4131: carriage moves left but not right	stuck left end contact – free it, it must spring back
XY 4131: drawing mirrored	flip the sign of XY_Y_DIR
LED not blinking, monitor silent after the start-up message, Command=L	firmware before 0.4.1 waited for the Atari when the SIO interface was disconnected – upload 0.4.1 (it then only reports "SIO Command trvale L" and USB works)
The wrong plotter draws	PL; prints the selected one, PL0507; / PL4131; switches

13. Additional programs: fractals and biorhythm chart

Besides lettering and HPGL files the program set can also draw **fractals**. Both programs go into the same folder as `plot.py`; with `--send` they send the drawing straight to the Arduino (homing with `HM` first), without it they only save an HPGL file and a PNG preview. They work for the Minigraf A 0507 and the XY 4131 – the figure is always scaled to the whole drawing area of the selected plotter.

13.1 fraktaly.py – ten fractals

```
py fraktaly.py --list                    list the fractals
py fraktaly.py TYPE [options]           creates fraktal.hpgl and fraktal.png
py fraktaly.py TYPE --send [options]    and plots it

py fraktaly.py koch --send
py fraktaly.py hilbert --order 5 --send
py fraktaly.py drak --speed 40 --send
py fraktaly.py strom --plotter 4131 --send
```

TYPE	Fractal	Default order	Strokes
`sierpinski`	Sierpiński triangle (arrowhead curve)	7 – 2187 segments of 1.7 mm	1
`koch`	Koch snowflake	4 – 768 of 2.4 mm	1
`hilbert`	Hilbert curve	6 – 4095 of 3.0 mm	1
`peano`	Peano curve	4 – 6560 of 2.4 mm	1
`drak`	dragon curve (Heighway)	12 – 4096 of 2.8 mm	1
`levy`	Lévy C curve	12 – 4096 of 2.1 mm	1
`gosper`	Gosper curve (flowsnake)	4 – 2401 of 3.5 mm	1
`ctverec`	quadratic Koch island	3 – 2048 of 1.8 mm	1
`strom`	fractal plant	5 – 1488 of 2.8 mm	512
`ker`	fractal bush	4 – 4096 of 4.8 mm	1024

Option	Meaning
`--order N`	order = level of detail; one more means roughly 2–4× more, shorter segments. Below 1 mm the program warns
`--plotter 0507 / 4131`	which plotter (default 0507); with --send it also switches the Arduino
`--fill`	stretch the figure over the whole area (distorts the shape, e.g. the triangle is no longer equilateral)
`--margin MM`	extra margin in mm (default 0 = whole drawing area)
`--speed MM/S`	drawing speed; slower (e.g. 40) helps with sharp corners
`--send`	plot right away (otherwise file and preview only)
`--port COM5`	other port than the default COM4
`-o file.hpgl`	output file name (the preview gets the same name .png)

13.2 sierpinski.py – Sierpiński triangle

A stand-alone version of the first fractal with the same options as `fraktaly.py` (without the type name):

```
py sierpinski.py --send              order 7, equilateral, 190 x 221 mm on the Minigraf
py sierpinski.py --order 6 --send    coarser and faster (729 segments)
py sierpinski.py --fill --send       over the whole area 190 x 268 mm
```

13.3 Tips

- While plotting, plot.py shows the elapsed and estimated remaining time, and the total plotting time at the end.
- Curves with thousands of segments take tens of minutes – every corner means the motors accelerate and decelerate. For a first try `koch` (768 segments) or `sierpinski --order 6` is the fastest.
- Draw the denser curves (Hilbert, Peano) with a thinner pen (0.3 mm), otherwise the lines merge.
- Always look at the preview `fraktal.png` first – it shows the exact position on the paper.
- If a command or a reply gets lost on USB, plot.py detects it and the drawing continues without an extra line; details are in plot_log.txt.

13.4 kondiciogram.py – biorhythms like on the 8-bits

The "kondiciogram" (biorhythm chart) was a popular 8-bit computer program: from the date of birth it computes three biorhythms and plots them for a chosen month, with a day-by-day table of signs below the chart. (Biorhythms are 1980s fun – they were never confirmed scientifically.) The chart texts are in Czech.

Curve	Biorhythm	Cycle	Line
F	physical – strength, stamina, health	23 days	solid
P	emotional – mood, feelings, creativity	28 days	dashed
I	intellectual – memory, logic, concentration	33 days	dotted

Sign	Meaning in the table
+	positive phase – curve above the axis, days full of energy
-	negative phase – curve below the axis, rest and recovery
*	critical day – the curve crosses the axis during the day, a day of uncertainty

```
py kondiciogram.py 12.5.1965                current month (file + preview)
py kondiciogram.py 12.5.1965 11/2026 --send  chosen month, plot right away
py kondiciogram.py 12.5.1965 --jmeno "Zdeněk Kotrba" --send
py kondiciogram.py 12.5.1965 --pera --send   each curve with a different pen
py kondiciogram.py 12.5.1965 --plotter 4131 --send
```

Option	Meaning
DATE	date of birth (12.5.1965, 12-5-1965 or 12/5/1965)
MONTH/YEAR	which month to plot, e.g. 11/2026; current month if omitted
--jmeno "…"	name in the header
--pera	text and grid first, then each curve separately; before each one plot.py lifts the pen and waits for a pen change and Enter
--plotter, --send, --port, --speed, -o	same as for fraktaly.py

In the current month, today is marked with a vertical line; Sundays are dotted. The program also prints the signs on the command line so you see the result at once.

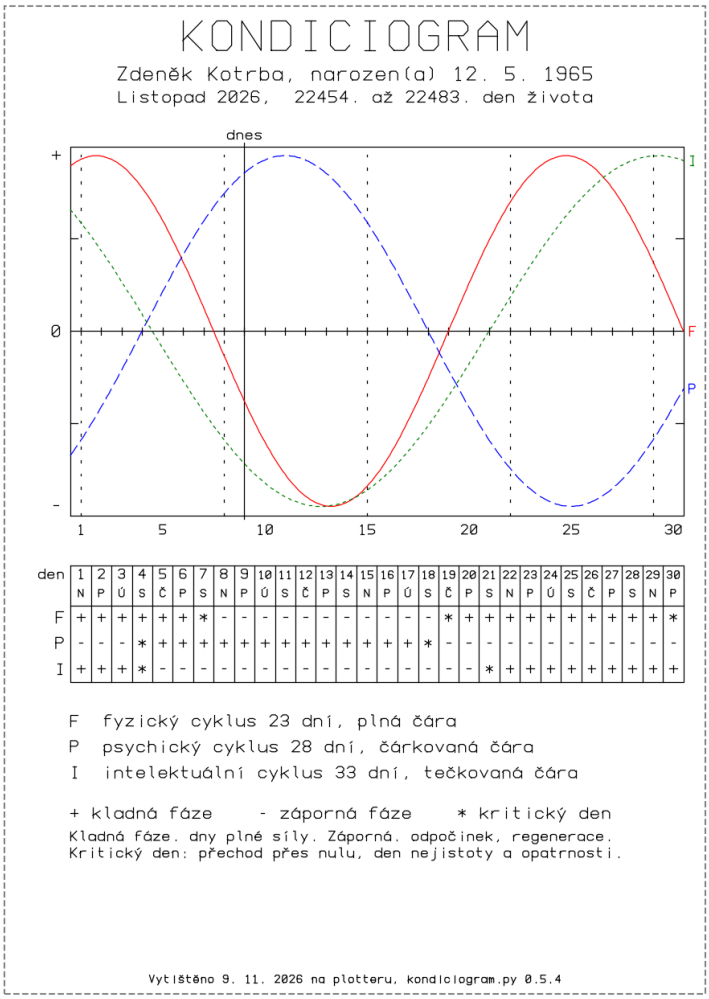


*Fig. 8 – Biorhythm chart preview (colours only in the preview; with **--pera** you draw them with different pens)*

14. Project files

File	Content
`firmware/minigraf_sio/minigraf_sio.ino`	firmware for Arduino Nano Every and Leonardo/Micro (Minigraf A 0507 and XY 4131)
`firmware/xy4131_test/xy4131_test.ino`	test program for bringing up the XY 4131
`pc/plot.py`	sending HPGL files from a PC
`pc/text_plot.py`	lettering, text in lines and test page
`pc/sierpinski.py`	Sierpiński triangle in one stroke (demo; py sierpinski.py --send)
`pc/fraktaly.py`	ten fractals (Koch, Hilbert, dragon, Gosper, plant …), most in one stroke
`pc/kondiciogram.py`	biorhythm chart (kondiciogram) for a chosen month
`pc/hershey_script.py`	script font data for text_plot.py --font psane
`docs/`	manuals (CZ, EN), schematics and photos

Note: the firmware and PC programs print their messages in Czech. Connector pinout based on cygnus.speccy.cz (Aritma Minigraf 0507); signal meaning from the MZXR ZX Spectrum documentation and the MAME emulator. XY 4131 wiring from the XY 4130 schematic and schotek.cz, verified by measurement.